Section 1.1 - Multiprocessing

The serial execution time for the chunked Mandelbrot implementation took 4.49 seconds when a resolution of 8192x8192 with a max_iter of 100 was used. The sweep was run with the same resolution as the serial implementation from 2 to 16 cores, of which the last 8 were logical. The timing was the median of 3 runs.

As can be seen in Table 1, and on Figure 1 and Figure 2, runs with an odd number of cores seems to perform worse than runs with an even number of cores. This is because 8192 cannot be neatly divided into odd chunks of the same size, and due to how the mandelbrot_parallel function partitions it results in a very small straggler task that one of the available cores must compute.

For three cores the partitioning mechanism results in the following number of rows: 2730, 2730, 2730, 2, where the last partition has a large amount of overhead compared to the amount of compute. This could be fixed by simply adding the fourth partition to the third to avoid the overhead extra overhead.

Number of cores	Time [s]	Speedup	Efficiency
1	4.4918	0	1.0
2	2.6497	1.6952	0.8476
3	3.6115	1.2438	0.4146
4	2.3212	1.9351	0.4838
5	2.4860	1.8069	0.3614
6	1.8847	2.3834	0.3972
7	1.8886	2.3784	0.3398
8	1.5423	2.9125	0.3641
9	1.5078	2.9791	0.3310
10	1.3087	3.4323	0.3432
11	1.2587	3.5686	0.3244
12	1.1538	3.8930	0.3244
13	1.0874	4.1309	0.3178
14	1.0025	4.4808	0.3201
15	0.9591	4.6834	0.3122
16	0.9099	4.9368	0.3086

Table 1, The execution time, speedup and efficiency measured for 1 to 16 cores.

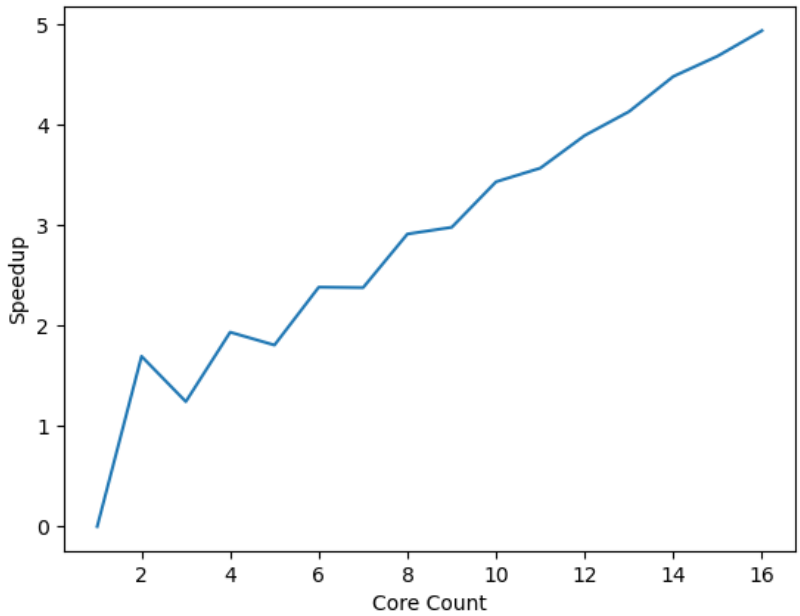

Figure 1, Speedup as a function of the number of cores.

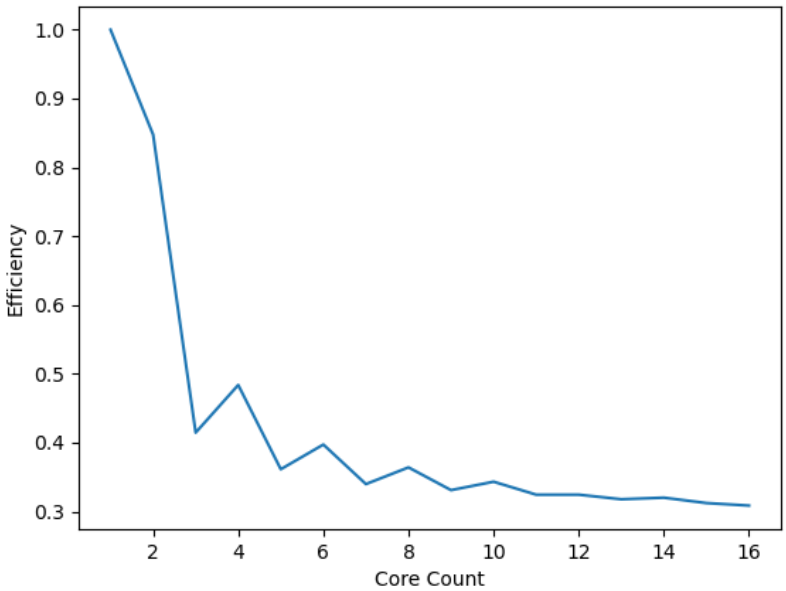

Figure 2, Efficiency as a function of the number of cores.

Both the speedup and efficiency graphs show that the perfect linear speedup is not achieved and with a total of 16 cores the effective speedup is just below 5x the serial implementation.

By looking at the smallest execution time we can calculate the implied serial fraction of the program using Equation 1.

$$s = \frac{\frac{1}{S_{p^*}} - \frac{1}{p^*}}{1 - \frac{1}{p^*}}$$

Equation 1

Where p^* indicate the number of processes / cores where the program execution time is the best. For the test results in Table 1, the best execution time at 0.9099 seconds is when 16 cores are used with a speedup of 4.9368. We can now calculate the implied serial fraction and we get the following:

$$s = \frac{\frac{1}{4.9368} - \frac{1}{16}}{1 - \frac{1}{16}} = 0.1494$$

We can now see that the implied serial fraction of the program is ~15%.

The code for this implementation can be found in “mandelbrot_mp.py”.

Section 1.2 – Multiprocessing Load Balanced

Table 2 shows the sweep results for the multi processed Mandelbrot set program when the chunk size is dynamic. The sweep was performed with a Mandebrot set resolution of 8192x8192 with a max_iter of 100 and with 16 workers / cores as this was found to be the optimal number in L4. The timing was the median of 3 runs.

Number of chunks	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	2.9458	4.5060	1.5296	9.4560
8	2.4239	4.5060	1.8590	7.6069
16	0.9111	4.5060	4.9459	2.2335
32	0.5531	4.5060	8.1472	0.9639
64	0.6007	4.5060	7.5015	1.1329
128	0.6079	4.5060	7.4122	1.1586
256	0.7511	4.5060	5.9992	1.6670
512	0.7992	4.5060	5.6384	1.8377
1024	0.7623	4.5060	5.9111	1.7068
2048	0.7608	4.5060	5.9225	1.7016
4096	0.7876	4.5060	5.7215	1.7965
8192	0.8720	4.5060	5.1672	2.0965

Table 2, Parallel execution time, serial time, speedup, LIF and implied serial portion measured for 4 to 8192 number of chunks.

By looking at Table 2, we can see that the best result is when splitting the set into 32 row chunks. We can now back-solve the implied serial fraction using Equation 1 in Section 1.1. We get the following:

$$s = \frac{\frac{1}{8.1472} - \frac{1}{16}}{1 - \frac{1}{16}} = 0.0643$$

We can now see that the implied serial fraction of the program is ~6.4%, this means that the chunked multi processed program improves the load balancing by 8.6 percent points. Likewise, the Load imbalance factor has also decreased by a lot and is now just 96%.

The implementation of the Mandelbrot set with Numba and parallel=True with a resolution of 8192x8192, takes 1.1681 seconds, which means that the load balanced multi-processed solution has a speedup slightly above 2 when comparing against Numba.

The code for this implementation can be found in “mandelbrot_mp_chunked.py”.

Section 2 – Dask Local

Sweeping was performed on a Mandelbrot set with a resolution of 8192x8192 with a max_iter of 100 from 4 chunks to 8192 chunks with a doubling of the chunk size each loop. The timing was recorded using the median of 3 runs using 8 workers, as this is the number of physical cores my AMD Ryzen 7 4800HS processor has. It should be noted that this is slightly different from the multiprocessing implementation which also used the logical cores. The reason for this is that Dask did not seem to utilize the logical cores and the parallel time for 128 chunks with 16 cores was 2.97 seconds which is nearly the same as with just the 8 physical cores as shown below. Table 2 lists the metrics that were recorded during the sweep.

Number of chunks	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	3.8483	4.7458	1.2464	5.4184
8	3.0660	4.7458	1.5645	4.1136
16	3.1897	4.7458	1.5038	4.3199
32	2.7975	4.7458	1.7146	3.6659
64	2.8578	4.7458	1.6784	3.7664
128	2.7796	4.7458	1.7256	3.6360
256	3.1799	4.7458	1.5084	4.3036
512	4.4055	4.7458	1.0772	6.3477
1024	7.2160	4.7458	1.0888	11.0353
2048	13.3141	4.7458	0.6647	21.2060
4096	28.5489	4.7458	0.3603	46.6154
8192	60.6467	4.7458	0.0791	100.1497

Table 3, Parallel execution time, serial time, speedup and LIF measured for 4 to 8192 number of chunks.

Figure 3 shows time as a function of the number of chunks.

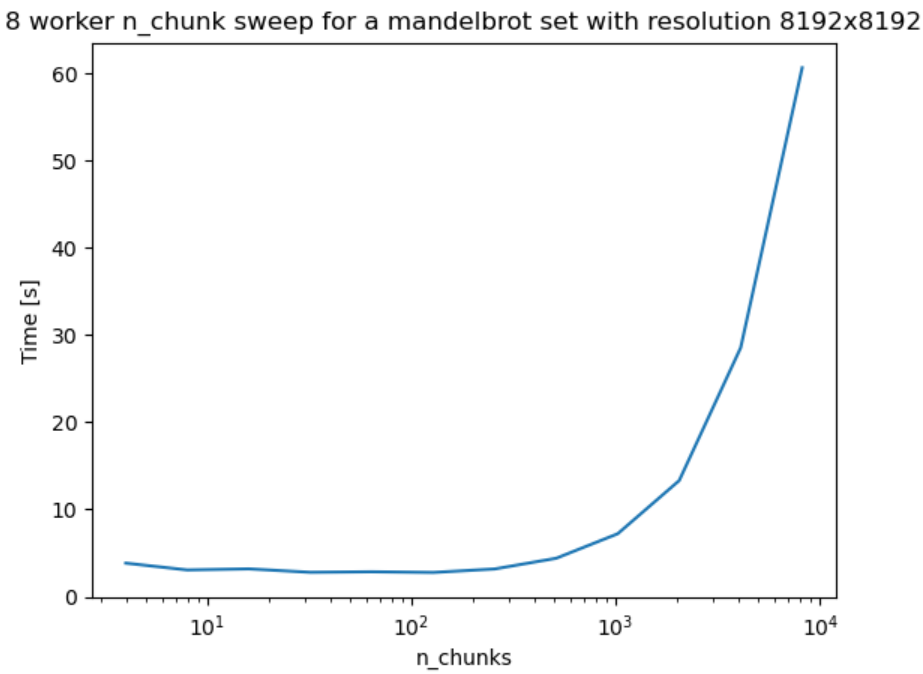

Figure 3, Time as a function of the number of chunks.

By observing Table 3 and looking at Figure 3 we can conclude that minimum execution time and Load Imbalance Factor (LIF) are 2. 7795s and 364% respectively. Therefore, the optimal number of chunks is 128 with each chunk being 8192x64 pixels in size, when the resolution of the Mandelbrot set is 8192x8192 and 8 workers are used.

It is also quite clearly seen that at and above 2048 chunks the overhead becomes too great and the execution time becomes slower than the serial implementation.

The code for this implementation can be found in "mandelbrot_dask_sweep.py".

How does Dask local compare to multiprocessing at the same worker count?

The multi-processed implementation from L4 takes 1.5423 seconds when 8 workers are used. This is a little faster than the Dask implementation of 2.7795 seconds with the same number of workers. However, multi-processed implementation can only be used locally on single machine while Dask enables distributed computing across multiple machines on a network.

What does the overhead difference tell you about when to choose each tool?

The overhead is clearly larger when using Dask, which is why there is a little more than 1 second difference between the implementations. This makes sense as Dask must inherently have more overhead to accommodate distributed computing across multiple networked machines.

Is the scheduling overhead visible at small resolutions?

Yes definitely, computing 1024x1024 using local Dask takes almost 0.1782 seconds, while the serial median with NJIT takes 0.0717 seconds and the multiprocessing implementation 0.0171 seconds. This means that the overhead for Dask is quite large.

Does performance improve as expected at larger N?

Yes, it is not as bad. As described above computing the Mandelbrot set with a resolution of 1024x1024 took 0.1782 seconds. By looking at Table 3 we can see that computing the Mandelbrot set with a resolution of 8192x8192 takes 2.7796 seconds. If increasing N did not increase performance the execution time should be $0.1782 \cdot 64 = 11.4048$ seconds.

Section 3 – Dask Distributed

Experiment 1 (Chunk sweep):

In the first experiment the number of chunks was swept from 4 to 8192. The Mandelbrot resolution was 8192x8192 with a max_iter of 100. The number of workers was set to 3 (AAU.CPU.b.2-4) nodes, each with 2 VCPUs, totalling 6 distributed workers, see Figure 4. The timing was the median of 3 runs, the results for the chunk sweep are shown in Table 4.

Number of chunks	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	3.5470	5.0063	1.4114	3.2511
8	2.8578	5.0063	1.7518	2.4251
16	2.0100	5.0063	2.4907	1.4090
32	2.2222	5.0063	2.2529	1.6633
64	1.8097	5.0063	2.7663	1.1690
128	1.7903	5.0063	2.7963	1.1457
256	1.7715	5.0063	2.8261	1.1231
512	1.8758	5.0063	2.6689	1.2481
1024	2.4077	5.0063	2.0792	1.8857
2048	3.8253	5.0063	1.3087	3.5847
4096	7.4932	5.0063	0.6681	7.9806
8192	16.7568	5.0063	0.2988	19.0830

Table 4, Parallel execution time, serial time, speedup and LIF measured for 4 to 8192 number of chunks with 6 distributed workers.

When compared to the local Dask it is quite clear that the CPUs are more powerful on AAU Strato, as just 6 distributed workers performed almost twice as well as the 8 local workers on my laptop, see Table 3. The optimal number of chunks was slightly larger than for the local Dask implementation, and therefore the chunk size is slightly smaller: 8192x32.

name	address	nthreads	cpu	memory	limit
Total (6)	(null)	6	0.2	1.6 GiB	11.5 GiB
tcp://10.92.0.121:402	tcp://10.92.0.121:40277	1	2 %	279.2 MiB	1.9 GiB
tcp://10.92.0.121:449	tcp://10.92.0.121:44985	1	2 %	267.9 MiB	1.9 GiB
tcp://10.92.0.174:360	tcp://10.92.0.174:36035	1	4 %	266.5 MiB	1.9 GiB
tcp://10.92.0.174:463	tcp://10.92.0.174:46351	1	2 %	266.1 MiB	1.9 GiB
tcp://10.92.0.250:329	tcp://10.92.0.250:32985	1	2 %	274.9 MiB	1.9 GiB
tcp://10.92.0.250:402	tcp://10.92.0.250:40225	1	4 %	260.4 MiB	1.9 GiB

Figure 4, the Dask dashboard showing the 6 distributed workers.

The code for this implementation can be found in “mandelbrot_dask_distributed_sweep.py”.

Experiment 2 (Worker sweep):

In the second experiment the Mandelbrot resolution was kept at 8192x8192 with a max_iter of 100. The number of chunks was kept at 256 (chunk size of 8192x32) and the number of workers was swept from

1 to 4 (AAU.CPU.d.4-12), each with 4 VCPUs, resulting in 4 – 18[1] distributed workers. The timing was the median of 3 runs, the results for this experiment are shown in Table 5.

Workers	Parallel Time [s]	Serial time [s]	Speedup	LIF
4	2.6260	5.0130	1.9090	1.0954
8	1.9611	5.0225	2.5611	2.1236
12	1.5739	5.0271	3.1940	2.7570
16	1.5057	5.0615	3.3615	3.7598
18	1.4661	5.0100	3.4173	4.2673

Table 5, Results of sweeping over the number of workers when the chunk size is kept static.

It should be noted that the last timing (18 workers), was done using a single instance of a AAU.CPU.c.2-8 node, as it seems Strato has a maximum quota for students that is 20, and since one node was already used for the Dask scheduler, no more could be used as workers.

For fun a single AAU.CPU.16-32 node with 16 VCPUs was started to see if it made had made a big difference that the work had to be distributed to 5 different nodes instead of just 1. As excepted, this had little impact and achieved a time that was a little slower than using a cluster with 4 AAU.CPU.d.4-12 nodes, see Table 6.

Workers	Parallel Time [s]	Serial time [s]	Speedup	LIF
16	1.5230	5.0160	3.2934	3.8582

Table 6, Results for using one AAU.CPU.16-32 node in a Dask distributed cluster.

The code for this implementation can be found in “mandelbrot_dask_distributed.py”.

Does speedup scale as expected with worker count?

No, we see a speedup, but it is not ideal. It is possible that this is due to the overhead Dask has, or a large serial part in my implementation.

Reflection MP2

The reflections have been included here for completeness.

Multiprocessing vs. Dask local vs. cluster - when is each the right tool?

Multiprocessing makes sense if the amount to compute is not too big and the transfer latency therefore has a huge impact on the timing. An example where this might matter is a critical system, where it might be slower to compute something locally on 2 or 4 cores, instead of computing it on an

[1] An additional AAU.CPU.c.2.8 node with 2 VCPUs was also used for the last result (18 workers).

edge node or in the cloud. However, due to the transfer latency, it might still be faster to compute locally as the total time including the transfer latency might be larger for the edge node and the cloud.

Local Dask has a larger overhead than multiprocessing, but provides a nice dashboard to show the status of the task, worker load, etc. I imagine I would use Dask local if I wanted this information, but generally I would prefer multiprocessing.

A Dask distributed cluster makes sense when the computation is large enough such that the transfer latency does not impact the timing. Another use case is if we have many nodes with 1 or 2 cores, separately these might simply not be able to perform the large computation fast enough, but as a distributed cluster their combined computational resources are pooled, and the large computation can be performed.

The most surprising result was probably that the parallel time scaled so badly with the number of workers, but as mentioned previously this might be a large serial part in the implementation.

Performance table

In Table 7 and Table 8, the updated performance tables are shown.

1024x1024	Time [s]	Speedup	Workers
Naïve	39.858	0	1
Vectorized	1.2566	31.7	1
NJIT Hybrid	1.1809	33.8	1
NJIT	0.1699	234.6	1
NJIT Parallel	0.0191	2086.8	8
Multiprocessing	0.0171	2330.9	16
Dask Local	0.1782	223.7	8

Table 7, Performance table for the Mandelbrot set with a resolution of 1024x1024.

8192x8192	Time [s]	Speedup	Workers
NJIT Parallel	1.1353	0	1
Multiprocessing	0.5531	2.0526	16
Dask Local	2.7796	0.4084	8
Dask Distributed	1.7715	0.6409	16

Table 8, Performance table for the Mandelbrot set with a resolution of 8192x8192.